

Folder and File Management

Feature Overview

Field	Value
Feature name	Folder and File Management
Status	Phase 1 complete; Phase 2 complete (custom icon graphics deferred)
Phase	Phase 1 (basic) + Phase 2 (full)
Priority	High

Description

Users can create, rename, and delete folders to organise their scanned documents. The system provides a set of pre-created default folders and allows users to add their own. Documents can be viewed, moved between folders, searched, and sorted.

Terminology note (Build 14): the user-facing label for a folder is now “Vault” (plural “Vaults”) throughout the app. This is a display-only rename. The underlying data model, API routes, URLs and code identifiers still use “folder” for stability, so this doc continues to use “folder” when describing the implementation.

Phase 1 Requirements (Basic)

- [x] Pre-created default folders for new users (Personal, School, Medical, Work, Other)
- [x] Create new folder (custom name)
- [x] Save scanned document to a selected folder
- [x] View list of documents in a folder
- [x] View/preview a document (image viewer; PDF shown as an icon with open/download)

Phase 2 Requirements (Full)

- [x] Rename folders
- [x] Delete folders (with confirmation dialog)
- [x] Move documents between folders
- [x] Delete documents (with confirmation dialog)
- [x] Search documents (by name, scoped to the signed-in user)
- [x] Sort documents by date or name
- [x] Folder icons - built-in icon set live; custom graphics from Tracey deferred

Implemented Behaviour (Phase 2)

Folder icons

- Folders support a built-in icon picker offering a curated set of choices (folder, personal, education, medical, work, document, wallet, home, travel, vehicle).
- The icon can be chosen when creating a folder and changed later via the folder Rename dialog. The selection persists through the folder update endpoint.
- Custom uploaded folder icons (client-supplied graphics from Tracey) are deferred until those image assets are provided. When the assets arrive, they can be added as additional choices or as an upload option without changing the storage model.

Move, rename, delete

- Documents can be moved to any of the user's folders from the document view; the target folder ownership is verified server-side.
- Deleting a folder removes its documents and decrements the owner's storage usage.
- Rename is capped at 60 characters.

Search and sorting

- A dedicated search screen (`/search`) calls `/api/search` .
- Search is case-insensitive name matching, scoped to the signed-in user, with an optional folder filter.
- Sort options: date newest, date oldest, name A to Z, name Z to A.

PDF documents

- Documents saved as PDFs (multi-page scans) render a PDF placeholder icon in the folder grid and a PDF panel with an open/download action in the document view, rather than a broken image thumbnail.

Technical Notes

Database Schema

- Folders table: `id`, `user_id`, `name`, `icon`, `is_default`, `created_at`, `updated_at`.
- Documents table: `id`, `user_id`, `folder_id`, `name`, `cloud_storage_path` (S3), `mime_type`, `file_size`, `thumbnail_path`, `is_public`, `created_at`, `updated_at`.
- Default folders are created automatically on user registration.

Cloud Storage

- Document files are stored in Amazon S3 (US).
- Access is via time-limited signed URLs.
- The data export's document links route through the app's own domain via `/api/files/[id]` (added Build 13), which streams the file server-side so the storage host is never exposed. The route is session-gated and scoped by user ID. (The in-app document viewer still opens files via time-limited signed URLs directly.)

User Isolation

- All queries are filtered by `user_id`.
- No cross-user data access at any level.
- S3 paths are scoped to the user ID.

Storage Quota

- Each user has a storage limit (default 5GB). Uploads are checked against remaining quota and rejected with a clear error when the limit would be exceeded.

UI Considerations

- Mobile-first layout (designed for phone screens inside GoodBarber).
- Thumbnail grid for documents, list for recent items.
- Tap to preview, menu for actions.
- Empty-state messaging for new users.

Offline Support (deferred)

- Local caching, offline queueing and sync are not implemented yet. This remains a later-phase design goal.

Dependencies

- Scanner feature (provides documents to store)
- Authentication system (provides user ID)
- Database schema (folders and documents tables)
- Cloud storage setup (S3 bucket and API)

Per-vault fields and cross-field search (Build 25, Phase A)

Each vault carries an ordered set of field definitions: a stable key, a human label, a type (text, number, date, or choice list), and options for a choice list. The ten default vaults ship with tailored starter fields; custom vaults start from a generic set and can be edited freely.

- Managing fields: each vault's menu has "Manage fields" (available on default and custom vaults). Fields can be added, renamed, retyped, given choice-list options, and removed. Renaming keeps values already saved; removing a field hides its saved values only. A vault is capped at 20 fields.
- Capturing values: the scanner save step shows the selected vault's fields as inputs so details are captured when a document is saved.
- Viewing and editing: the document screen shows a Details panel with the vault's fields and their values (dates formatted) plus an "Edit details" action. Moving a document to another vault reloads its details against the destination vault's fields.
- Search: matches the document name and all saved field values. Each document keeps a derived lowercase search string (name plus field values) used for matching.
- Data model: `Folder.fields` (JSON), `Document.metadata` (JSON), `Document.searchText` (String). All additive. Field helpers live in `lib/vault-fields.ts`.

Preview thumbnails on the grid (Build 26)

Folder grid tiles now show a real preview image for documents that have one, instead of squeezing the full image into the portrait tile or showing a generic icon.

- Thumbnails are generated at save time (see `scanner.md`) and stored in `Document.thumbnailPath` (an existing field; no schema change). The whole page fitted on white is the default; the user can set a custom 3:4 "cover" frame from the save step.

- The folders list returns `hasThumbnail` per document so the grid knows whether to render an image tile. Tiles fetch the preview from `GET /api/files/[id]?variant=thumb`, which streams the thumbnail inline as a JPEG and falls back to the full file if none exists.
- Scanned images and scan-built PDFs get a real preview tile. Externally uploaded PDFs keep the generic PDF icon (no source image to render). Documents saved before this build have no thumbnail and keep their previous tile appearance.
- Deleting a document also removes its thumbnail from storage. Thumbnails are small and do not count against the storage quota.

Show/hide previews on the grid (Build 27)

A vault that contains documents shows an eye icon in its header. Tapping it hides all document image previews on that vault's grid (each tile shows a neutral "Preview off" placeholder); tapping again shows them. The choice is stored per browser (local storage key `lockondocs.showPreviews`) so it survives a reload and applies across vaults. When previews are off, the tile image element is not rendered at all, so no document image is requested.

This is both a privacy control (avoid rendering sensitive document images in the grid, for example over someone's shoulder) and a diagnostic aid (turn previews off, reload a populated vault, and check whether a browser security warning is being triggered by the rendered imagery). Client-only; no schema or API change.

Expiry reminders (Build 28, Phase B)

The app tracks each document's expiry or renewal date and surfaces documents that are expiring soon or have already expired. This is the in-app reminders layer; scheduled email reminders are a separate follow-on and are not part of this build.

- Where the date comes from: the app reads the vault's designated date field (an expiry, renewal or retention field from Phase A) and records the document's expiry date when the document is saved or its details are edited. No separate expiry input was added.
- Reminders screen: reached from the bell icon at the top left of the home screen. It lists documents that need attention, split into "Expired" and "Expiring soon", each with a plain-English label and a link to the document, and each dismissable.
- Home screen: shows a bell icon with a count badge and a banner when any documents need attention (both hidden when there is nothing to show).
- Document screen: shows a red (expired) or amber (expiring soon) status badge above the Details panel.
- Lead time: each user chooses how far ahead they are reminded (7, 14, 30, 60 or 90 days; default 30), set on the Reminders screen.
- Data model: `Document.expiryDate` (DateTime, nullable), `Document.reminderDismissedAt` (DateTime, nullable), `User.reminderLeadDays` (Int, default 30), plus an index on (userId, expiryDate). All additive. Logic lives in `lib/reminders.ts`; the vault expiry field is chosen by `getExpiryFieldKey` in `lib/vault-fields.ts`.
- APIs: `GET /api/reminders` (non-dismissed, expired or within-lead documents for the signed-in user, soonest first); `PATCH /api/reminders` (update lead time, or dismiss a document). All queries are scoped to the signed-in user.

Email expiry reminders (Build 29, Phase B follow-on)

Because the app runs embedded in the GoodBarber mobile shell, it cannot send phone push notifications. Branded reminder emails fill that gap so users hear about expiring documents without having to open the vault.

- What is sent: for each document, at most one “expiring soon” email (when it comes within 30 days of its expiry date) and, later, at most one “expired” email (once the date has passed). Exactly one of each per expiry cycle, so users are never spammed.
- Re-arming: if a document is renewed and its expiry date changes, both “sent” markers are cleared, so the new date can warn again with a fresh soon and expired email.
- Branding: the emails use the same branded template as the rest of the app (logo, colours, a clear call-to-action button that opens the document). GB English, no em dashes.
- Opt out: each user can turn email reminders off with a toggle on the Reminders screen. When off, that user is skipped entirely by the send job. A document whose in-app reminder has been dismissed is also skipped.
- Data model (additive, no data loss): `User.emailRemindersEnabled` (Boolean, default true), `Document.reminderSoonSentAt` (DateTime, nullable), `Document.reminderExpiredSentAt` (DateTime, nullable).
- How it runs: a protected endpoint, `POST /api/reminders/dispatch`, does the work. It is guarded by a secret (sent as an `Authorization: Bearer` header or an `x-cron-secret` header) and is designed to be called once a day by a scheduled task. Each run selects the documents that need an email, sends it, marks the matching “sent” flag only on success, and records the result in the email log. It reports counts (candidates, soon sent, expired sent, failed) and writes a `reminder_emails_dispatched` audit entry.
- Every sent or failed email is recorded in the admin Emails log (see `admin.md`).
- IMPORTANT: the daily send job only runs against the live host, so the reminder emails begin only after the build is deployed to `vault.lockondocs.app`.
- Build 30 boundary fix: the daily send job now includes a document that is exactly 30 days from expiry, matching the in-app bell exactly (previously a document exactly 30 days out lit the bell but produced no email).

Masked sensitive fields on the document screen (Build 31, Phase C)

Because vaults hold high-sensitivity details (ID numbers, passport and licence numbers, account and policy numbers, usernames, patient names), the document Details panel now masks those values by default so they are not exposed to someone glancing at the screen.

- What is masked: only text and number fields whose name matches a sensitive pattern (for example id number, passport, licence, SSN or social security, account number, VIN, tax id, policy number, username, password, recovery code, patient name). Ordinary descriptive fields (for example account type or a service name) stay visible.
- Reveal and hide: when a sensitive field has a value, the Details header shows a Reveal control. Revealing asks the user to confirm their password (via the re-authentication endpoint) and then shows the values; Hide re-masks them. Masked values render as bullet dots until revealed.
- Auto re-mask: if the app is backgrounded (the tab is hidden), revealed values are re-masked automatically, so nothing sensitive is left on screen when the user returns.

- This works together with the app-wide auto-lock and re-authentication described in `admin.md` and `production.md`.